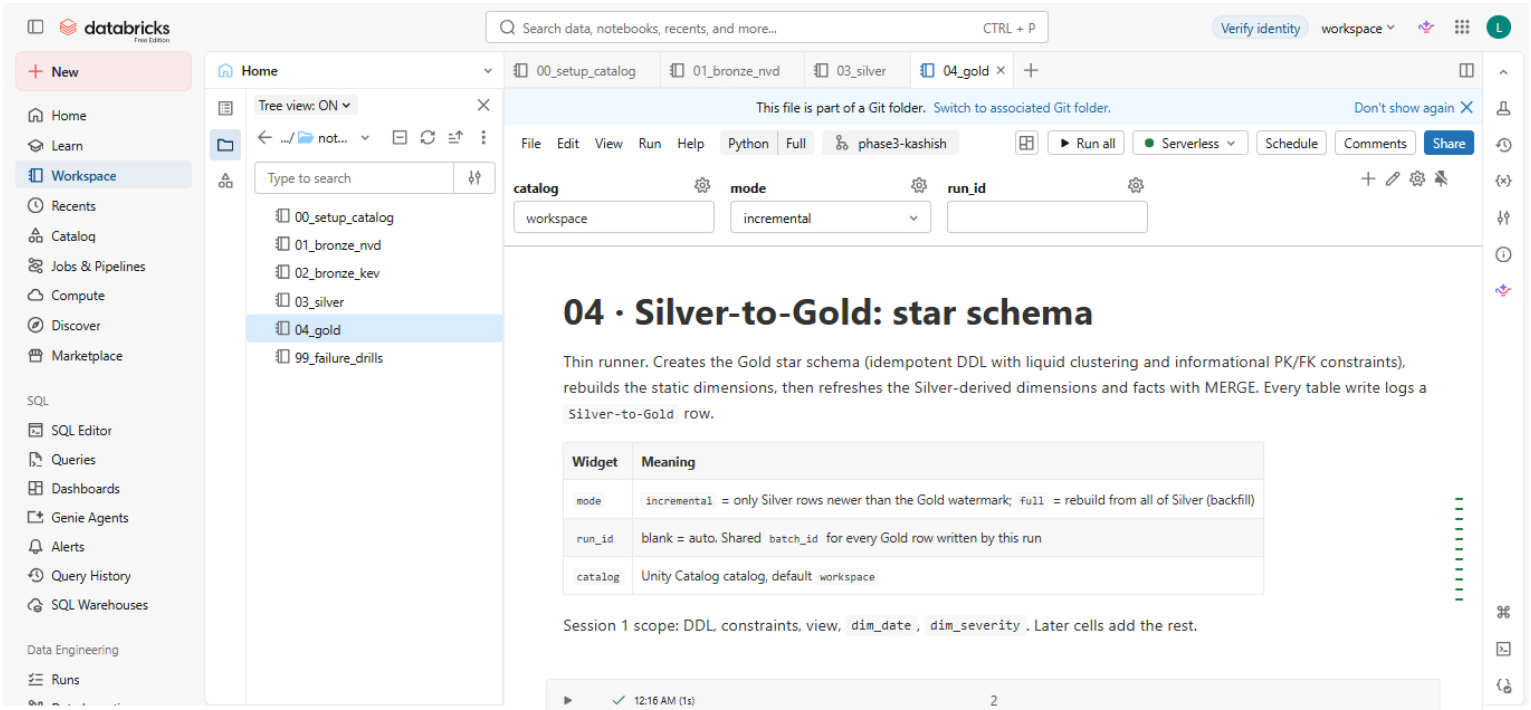


04 · Silver-to-Gold: star schema

Thin runner. Creates the Gold star schema (idempotent DDL with liquid clustering and informational PK/FK constraints), rebuilds the static dimensions, then refreshes the Silver-derived dimensions and facts with MERGE. Every table write logs a `Silver-to-Gold` row.

Widget	Meaning
`mode`	`incremental` = only Silver rows newer than the Gold watermark; `full` = rebuild from all of Silver (backfill)
`run_id`	blank = auto. Shared `batch_id` for every Gold row written by this run
`catalog`	Unity Catalog catalog, default `workspace`

Session 1 scope: DDL, constraints, view, `dim_date`, `dim_severity`. Later cells add the rest.

✓ 12:16 AM (1s) 2

```
%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
 %reload_ext autoreload
```

✓ 12:16 AM (<1s) 3

```
import os
import sys

SRC_PATH = os.path.abspath(os.path.join(os.getcwd(), "..", "src"))
if SRC_PATH not in sys.path:
    sys.path.insert(0, SRC_PATH)

from pyspark.sql import functions as F  # noqa: E402

from vulnpulse.audit import execution_log as log  # noqa: E402
from vulnpulse.gold import ddl, dims_static  # noqa: E402
from vulnpulse.utils.params import new_batch_id, utc_now  # noqa: E402

spark.conf.set("spark.sql.session.timeZone", "UTC")
```

✓ 12:16 AM (<1s) 4

```
dbutils.widgets.dropdown("mode", "incremental", ["incremental", "full"])
dbutils.widgets.text("run_id", "")
dbutils.widgets.text("catalog", "workspace")

MODE = dbutils.widgets.get("mode")
RUN_ID = dbutils.widgets.get("run_id").strip() or new_batch_id()
CATALOG = dbutils.widgets.get("catalog").strip() or "workspace"

GOLD = f"{CATALOG}.vulnpulse_gold"
SILVER = f"{CATALOG}.vulnpulse_silver"
LOG_TABLE = f"{GOLD}.{log.EXECUTION_LOG_TABLE}"

print("mode:  ", MODE)
print("run_id:", RUN_ID)
print("gold:  ", GOLD, "| silver:", SILVER, "| log:", LOG_TABLE)
```

```
mode:   incremental
run_id: 20261016T191618Z-0feb9b60
gold:   workspace.vulnpulse_gold | silver: workspace.vulnpulse_silver | log: workspace.vulnpulse_gold.pipeline_execution_logs
```

1. Schema: tables, constraints, view

✓ 12:16 AM (16s) 6

```
tables = ddl.ensure_tables(spark, GOLD)
for fqn in tables.values():
    print(f"table ready: {fqn}")

constraints = ddl.ensure_constraints(spark, GOLD)
for name, outcome in constraints.items():
    print(f"  constraint {name:<42} {outcome}")

print("view ready:", ddl.ensure_watchlist_view(spark, GOLD))
```

```
def log_gold_write(target: str, started_at, rows_read: int, counts: dict, error=None) -> None:
    """One Silver-to-Gold audit row per table write (contract rule 25)."""
    log.append(
        log.build_log_row(
            spark,
            batch_id=RUN_ID,
            layer=log.LAYER_SILVER_TO_GOLD,
            source=target,
            load_type="FULL" if MODE == "full" else "INCREMENTAL",
            parameters={"mode": MODE, "run_id": RUN_ID, "catalog": CATALOG},
            started_at=started_at,
            finished_at=utc_now(),
            status=log.STATUS_FAILURE if error else log.STATUS_SUCCESS,
            rows_read=rows_read,
            rows_inserted=counts.get("inserted", 0),
            rows_updated=counts.get("updated", 0),
            rows_quarantined=0,
            error_message=error,
        ),
        LOG_TABLE,
    )
```

See performance (31) 22 Checked for improvements

```
  constraint fk_fact_cve_last_modified_date_key        skipped: [RequestId=da80e7ae-3781-4d14-aaac-055963fdb4f0 ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_kev_added_date_key            skipped: [RequestId=5cb800e5-a589-4a65-bdc2-bc81cc02ddda ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_kev_due_date_key              skipped: [RequestId=c58e34a4-a735-4758-b1d7-a250077f7fed ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_severity_key                  skipped: [RequestId=c26f2739-6564-429c-ad8b-3010c694a31e ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_product_cve_id                exists
  constraint fk_fact_cve_product_vendor_product_key    exists
  constraint fk_fact_cve_product_published_date_key    skipped: [RequestId=b494fdbb-c4c5-4904-8b9d-f63107b316e8 ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_product_severity_key          skipped: [RequestId=3ca04cd9-d172-4a8d-ba22-225be5a2e35f ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_cwe_cve_id                    exists
  constraint fk_fact_cve_cwe_cwe_key                   exists
  constraint fk_fact_cve_cwe_published_date_key        skipped: [RequestId=d129a971-c54d-406a-a94a-d045e50e9bc6 ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
  constraint fk_fact_cve_cwe_severity_key              skipped: [RequestId=d404a352-af1a-4a58-b131-1ac0077b6d33 ErrorCla
ss=NOT_FOUND.UC_REFERENTIAL_CONSTR
view ready: workspace.vulnpulse_gold.gold_watchlist_v
```

2. Static dimensions (generated, overwritten every run)

▶ ✓ 12:16 AM (9s) 8

```
for name, builder in (
    ("dim_date", dims_static.build_dim_date),
    ("dim_severity", dims_static.build_dim_severity),
):
    started = utc_now()
    error = None
    written = 0
    try:
        df = builder(spark, batch_id=RUN_ID, load_timestamp=started)
        written = dims_static.overwrite(df, tables[name])
        print(f"{name}: {written} rows written")
    except Exception as exc:  # noqa: BLE001 - logged then re-raised
        error = f"{type(exc).__name__}: {exc}"[:2000]
        raise
    finally:
        log_gold_write(tables[name], started, written, {"inserted": written, "updated": 0}, error)
```

See performance (6) Checked for improvements

```
dim_date: 15706 rows written
dim_severity: 6 rows written
```

3. CVE core: `dim_cve` and `fact_cve` (MERGE, idempotent)

Reads `silver_cve` and `silver_kev`. If Zahra's `silver_affected_product` / `silver_cwe` tables exist yet, their per-CVE counts are attached; otherwise the count columns stay NULL. Session 3 adds the Gold watermark so only changed CVEs are processed in `incremental` mode.

▶ ✓ 12:16 AM (3s) 10

```
from vulnpulse.gold import cve as gold_cve  # noqa: E402
from vulnpulse.gold import merge as gold_merge  # noqa: E402

silver_cve = spark.table(f"{SILVER}.silver_cve")
silver_kev = spark.table(f"{SILVER}.silver_kev")


def optional_counts(table: str):
    fqn = f"{SILVER}.{table}"
    if spark.catalog.tableExists(fqn):
        print(f"child counts from {fqn}")
        return gold_cve.count_per_cve(spark.table(fqn))
```

```
    print(f"{fqn} not present yet; count column will be NULL")
    return None


product_counts = optional_counts("silver_affected_product")
cwe_counts = optional_counts("silver_cwe")

rows_in = silver_cve.count()
print(f"silver_cve rows read: {rows_in}; silver_kev rows: {silver_kev.count()}")
```

See performance (4) ✓ Checked for improvements

2 dataframes: silver_cve, silver_kev

```
workspace.vulnpulse_silver.silver_affected_product not present yet; count column will be NULL
workspace.vulnpulse_silver.silver_cwe not present yet; count column will be NULL
silver_cve rows read: 25777; silver_kev rows: 1734
```

▶ ✓ 12:16 AM (18s) 11

```
for name, build in (
    ("dim_cve", lambda ts: gold_cve.build_dim_cve(silver_cve, silver_kev, batch_id=RUN_ID, load_timestamp=ts)),
    (
        "fact_cve",
        lambda ts: gold_cve.build_fact_cve(
            silver_cve,
            silver_kev,
            batch_id=RUN_ID,
            load_timestamp=ts,
            product_counts=product_counts,
            cwe_counts=cwe_counts,
        ),
    ),
):
    started = utc_now()
    error = None
    counts = {}
    try:
        counts = gold_merge.merge_gold(spark, build(started), tables[name], keys=["cve_id"])
        print(f"{name}: read {rows_in}, inserted {counts['inserted']}, updated {counts['updated']}")
    except Exception as exc:  # noqa: BLE001 - logged then re-raised
        error = f"{type(exc).__name__}: {exc}"[:2000]
        raise
    finally:
        log_gold_write(tables[name], started, rows_in, counts, error)
```

See performance (8) Optimize

```
dim_cve: read 25777, inserted 0, updated 0
fact_cve: read 25777, inserted 0, updated 0
```

Results

▶ ✓ 12:16 AM (2s) 13

```
display(
    spark.sql(f"SHOW TABLES IN {GOLD}").select("tableName", "isTemporary").orderBy("tableName")
)
```

See performance (2) ✓ Checked for improvements

Table +

	tableName	isTemporary
1	_gold_merge_source	true
2	dim_cve	false
3	dim_cwe	false
4	dim_date	false
5	dim_severity	false
6	dim_vendor_product	false
7	fact_cve	false
8	fact_cve_cwe	false
9	fact_cve_product	false
10	gold_watchlist_v	false
11	pipeline_execution_lo...	false
12	pipeline_watermarks	false

12 rows | 1.645s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

▶ ✓ 12:16 AM (8s) 14

```
display(
    spark.sql(
        f"""
        SELECT s.severity, count(*) AS cves, sum(f.is_exploited) AS exploited,
               round(avg(f.cvss_v3_score), 2) AS avg_score
        FROM {GOLD}.fact_cve f JOIN {GOLD}.dim_severity s ON s.severity_key = f.severity_key
        GROUP BY s.severity, s.sort_order ORDER BY s.sort_order
        """
    )
)
display(
    spark.sql(
        f"""
```

```
        SELECT count(*) AS fact_rows, count(DISTINCT cve_id) AS distinct_cves,
               sum(is_exploited) AS in_kev, sum(is_ransomware) AS ransomware,
               percentile_approx(days_to_kev, 0.5) AS median_days_to_kev,
               min(days_to_kev) AS min_days_to_kev, max(days_to_kev) AS max_days_to_kev
        FROM {GOLD}.fact_cve
        """
    )
)
display(spark.sql(f"SELECT count(*) AS watchlist_rows FROM {GOLD}.{ddl.WATCHLIST_VIEW}"))
```

See performance (3) Optimize

Table

	severity	cves	exploited	avg_score
1	CRITICAL	3421	78	9.72
2	HIGH	9807	96	7.92
3	MEDIUM	9742	23	5.63
4	LOW	555	1	3.2
5	UNKNOWN	2252	0	null

5 rows | 7.622s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

Table

	fact_rows	distinct_cves	in_kev	ransomware	median_days_to_kev	min_days_to_kev
1	25777	25777	198	51	555	-2

1 row | 7.622s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

Table

	watchlist_rows
1	457

1 row | 7.622s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

12:16 AM (2s) 15

```
display(
    spark.table(tables["dim_date"])
    .groupBy("year")
    .agg(F.count("*").alias("days"), F.sum(F.col("is_weekend").cast("int")).alias("weekend_days"))
    .orderBy("year")
    .limit(5)
)
display(spark.table(tables["dim_severity"]).orderBy("sort_order"))
```

See performance (2) Checked for improvements

Table

	year	days	weekend_days
1	1988	366	105
2	1989	365	105
3	1990	365	104
4	1991	365	104
5	1992	366	104

5 rows | 2.189s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

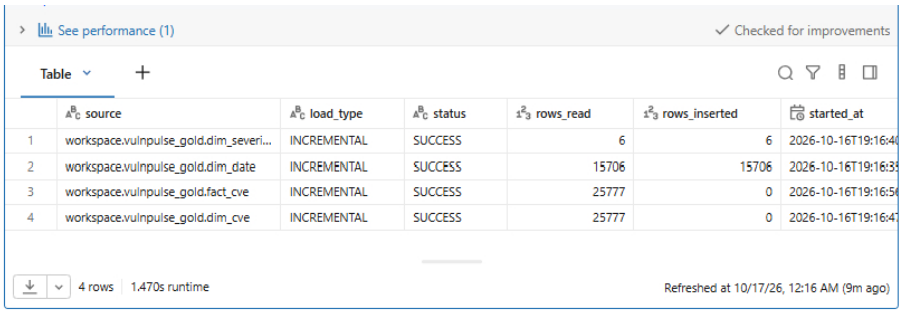
Table

	severity_key	severity	score_min	score_max	sort_order	description
1	1	CRITICAL	9	10	1	CVSS v3 base score 9.0-10.0
2	2	HIGH	7	8.9	2	CVSS v3 base score 7.0-8.9
3	3	MEDIUM	4	6.9	3	CVSS v3 base score 4.0-6.9
4	4	LOW	0.1	3.9	4	CVSS v3 base score 0.1-3.9
5	5	NONE	0	0	5	CVSS v3 base score 0.0
6	6	UNKNOWN	null	null	6	No CVSS v3 metric published (v2-or

6 rows | 2.189s runtime Refreshed at 10/17/26, 12:16 AM (9m ago)

12:16 AM (1s) 16 Python

```
display(
    spark.table(LOG_TABLE)
    .where(F.col("layer") == log.LAYER_SILVER_TO_GOLD)
    .where(F.col("batch_id") == RUN_ID)
    .select("source", "load_type", "status", "rows_read", "rows_inserted", "started_at", "finished_at")
)
```



See performance (1)

Checked for improvements

Table

	source	load_type	status	rows_read	rows_inserted	started_at
1	workspace.vulnpulse_gold.dim_severi...	INCREMENTAL	SUCCESS	6	6	2026-10-16T19:16:40
2	workspace.vulnpulse_gold.dim_date	INCREMENTAL	SUCCESS	15706	15706	2026-10-16T19:16:35
3	workspace.vulnpulse_gold.fact_cve	INCREMENTAL	SUCCESS	25777	0	2026-10-16T19:16:56
4	workspace.vulnpulse_gold.dim_cve	INCREMENTAL	SUCCESS	25777	0	2026-10-16T19:16:47

4 rows | 1.470s runtime

Refreshed at 10/17/26, 12:16 AM (9m ago)

Data Ingestion